Constraints in RDBMS

Agenda

1. Constraints in RDBMS
2. Using constraints
3. Implementing key
4. Cascade and set null
5. Auto increment
6. Composite key
7. Altering constraints
8. Summary

1. Constraints in RDBMS

OBJECT™ TECHNOLOGIES

- Constraints are the rules imposed on tables so that the data entered in the table is a correct data.
  - Constraints are used to limit the type of data that can go into the table.
  - Ensures the accuracy and reliability of the data in the table.
  - Constraints are used, so that the data though scattered in multiple tables remain consistent and integrated.
  - Constraints are also called as data integrity constraints.

- Constraints and their need
  - Prevent the deletion/updation of a table if there are dependencies from other table.
  - Data consistency and integrity

- Types of constraints

  Column Level Constraints

  Table Level Constraints

- Data Integrity constraints

Not Null	Unique
Primary Key	Foreign Key
Check	Default

2. Using constraints

- **NOT NULL**

```
sql> CREATE TABLE Student
     (
       ID int  NOT NULL,
       NAME varchar(10)  NOT NULL,
       ADDRESS varchar(20)
     );
```

- **UNIQUE**

```
sql> CREATE TABLE Student
     (
       ID int  NOT NULL UNIQUE,
       NAME varchar(10),
       ADDRESS varchar(20)
     );
```

OR

```
sql> CREATE TABLE Persons
     (
       ID int UNIQUE,
       LastName varchar(20) NOT NULL,
       FirstName varchar(20),
       Age int,
       CONSTRAINT UC_Person_ID UNIQUE (ID)
     );
```

UC_Person is the constraint name.

- **Check Constraint**

```
sql> CREATE TABLE Persons
     (
       ID int NOT NULL,
       LastName varchar(25 ) NOT NULL,
       FirstName varchar(25 ),
       Age int CHECK (Age>=18) ,
       City varchar(25)
     );
```

OR

```
sql> CREATE TABLE Persons
     (
       ID int NOT NULL,
       LastName varchar(25) NOT NULL,
       FirstName varchar(25),
       Age int,
       City varchar(25),
       CONSTRAINT CHK_Person CHECK (Age>=18 )
     );
```

CHK_Person is a constraint name.

2. Using constraints

- **Default Constraint**

```
sql> CREATE TABLE Student
     (
        ID int NOT NULL,
        NAME varchar(10) NOT NULL,
        AGE int DEFAULT 18
     );
```

OBJECT™ TECHNOLOGIES

- **PRIMARY KEY**

```
sql> CREATE TABLE Student
  (
   ID int NOT NULL UNIQUE,
   NAME varchar(10),
   ADDRESS varchar(20),
   PRIMARY KEY(ID)
  );
```

or

```
sql> CREATE TABLE Student
  (
   ID int ,
   NAME varchar(10),
   ADDRESS varchar(20),
   constraint PK_ID PRIMARY
KEY(ID);
  );
```

PK_ID is a constraint name.

- **FOREIGN KEY**

```
sql> CREATE TABLE orders
  (
     o_id int not null,
     order_no int not null,
     c_id int not null,
     primary key(c_id)
  );
sql> CREATE TABLE customers
  (
     C_ID int not null,
     name varchar(20) NOT NULL,
     adderss varchar(20) NOT NULL,
   FOREIGN KEY (C_ID) REFERENCES orders(C_ID)
  );
```

or

```
CREATE TABLE customers
(
  C_ID int not null,
  name varchar(20) NOT NULL,
  adderss varchar(20) NOT NULL,
  constraint FK_ID FOREIGN KEY (C_ID) REFERENCES
orders(C_ID)
);
```

FK_ID is a constraint name.

4. Cascade and set null

- ON DELETE CASCADE

- create table deptTab
```
(
  deptno int,
  dname varchar(15),
  constraint pk_dno primary key(deptno)
);
```

- create table empTab
```
(
  empno int,
  ename varchar(15),
  constraint pk_eno primary key(empno),
  constraint fk_dno foreign key(deptno)
  references deptTab(deptno)
  ON DELETE CASCADE
);
```

- ON DELETE SET NULL

- create table deptTab
```
(
  deptno int,
  dname varchar(15),
  constraint pk_dno primary key(deptno)
);
```

- create table empTab
```
(
  empno int,
  ename varchar(15),
  constraint pk_eno primary key(empno),
  constraint fk_dno foreign key(deptno)
  references deptTab(deptno)
  ON DELETE SET NULL
);
```

4. Cascade and set null

OBJECT™ TECHNOLOGIES

ON UPDATE CASCADE

- create table deptTab
  ```
  (
    deptno int,
    dname varchar(15),
    constraint pk_dno primary key(deptno)
  );
  ```

- create table empTab
  ```
  (
    empno int,
    ename varchar(15),
    constraint pk_eno primary key(empno),
    constraint fk_dno foreign key(deptno)
    references deptTab(deptno)
    ON UPDATE CASCADE
  );
  ```

ON UPDATE SET NULL

- create table deptTab
  ```
  (
    deptno int,
    dname varchar(15),
    constraint pk_dno primary key(deptno)
  );
  ```

- create table empTab
  ```
  (
    empno int,
    ename varchar(15),
    constraint pk_eno primary key(empno),
    constraint fk_dno foreign key(deptno)
    references deptTab(deptno)
    ON UPDATE SET NULL
  );
  ```

5. Auto increment

- AUTO_INCREMENT attribute is used to generate a unique identity for new rows.

- Example

```
sql> CREATE TABLE t1 (
        c1 INT(11) NOT NULL AUTO_INCREMENT,
        c2 VARCHAR(10) DEFAULT NULL,
        PRIMARY KEY (c1)
     ) ;
```

6. Composite key

- Combination of multiple columns.

Example

```
sql>create admission
      ( sid int,
        cid int,
        CONSTRAINT cpy_admission PRIMARY KEY (SID,CID)
);
```

7. Altering constraints

OBJECT TECHNOLOGIES™

Adding NOT NULL constraint

```
ALTER TABLE Person
MODIFY age int NOT NULL;
```

Adding CHECK constraint

```
ALTER TABLE Persons
ADD CHECK (Age>=18);
or
ALTER TABLE Persons
ADD CONSTRAINT CHK_PersonAge CHECK (Age>=18);
```

Adding DEFAULT constraint

```
ALTER TABLE Persons
ADD DEFAULT city='PUNE';
or
ALTER TABLE Persons
ADD def_city DEFAULT city='PUNE';
```

Adding UNIQUE constraint

```
ALTER TABLE Persons
ADD CONSTRAINT uni_empno UNIQUE (empno);
or
ALTER TABLE Persons
ADD  UNIQUE(empno);
```

Adding Primary Key constraint

```
ALTER TABLE Persons
ADD PRIMARY KEY (ID);
or
ALTER TABLE Persons
ADD CONSTRAINT PK_Person PRIMARY KEY (ID );
```

Adding Foreign Key constraint

```
ALTER TABLE Orders
ADD FOREIGN KEY (PersonID) REFERENCES
                                Persons(PersonID);
or
ALTER TABLE Orders
ADD CONSTRAINT FK_PersonOrder
FOREIGN KEY (PersonID) REFERENCES
                                Persons(PersonID);
```

8. Summary

1. Constraints in RDBMS
2. Using constraints
3. Implementing key
4. Cascade and set null
5. Auto increment
6. Composite key
7. Altering constraints